Note that some shells treat ~ as a special character, and try to expand it to a directory name. You can avoid this issue by quoting the tag name:

```
$ rspec --tag '~fast'
```

The previous examples look for the tag's *truthiness*; any value besides nil or false will match. Sometimes, you care about the specific tag value. For example, you may have several specs tagged with a bug ID from your bug tracking system. If you run RSpec like so:

```
$ rspec --tag bug_id:123
```

...you can filter the examples to just the ones related to the ticket you're working on.

Sharing Code Conditionally

In *Structuring Code Examples*, we discussed three ways to share code across many example groups:

- Top-level config hooks
- Modules containing helper methods
- Shared contexts containing RSpec constructs (such as hooks and let blocks)

You've used all of these techniques throughout this book. By default, they all share code unconditionally. If you define, say, a before hook in your RSpec.configure block, the hook will run for *every* example.

Often, though, you want to use a certain bit of shared code only for specific examples. For instance, in *Isolating Your Specs Using Database Transactions*, on page 92, you defined an around hook to wrap a database transaction around only the examples tagged with :db:

06-integration-specs/07/expense_tracker/spec/support/db.rb

```
c.around(:example, :db) do |example|
  DB.transaction(rollback: :always) { example.run }
end
```

Metadata is what enables this flexibility, and you can use it with all of the code-sharing techniques listed earlier. Here's how:

Config hooks

Pass a filter expression as the second argument to config.before, config.after, or config.around to run that hook only for examples matching the filter.